

```

// SetupFunctions.h

#include "SoftwareSerial.h"
#define NODEMCU_FILE

#include <stdint.h>

/** @file SetupFunctions.h
 * @brief Sets Up WiFi and other networking features
 */

#pragma once

#define HTTP_PORT 80

/**
 * @brief Sets up the WiFi on the NodeMCU.
 *
 * @details Iterates through "pass_list" (defined in sensitive_data.c)
 *
 * @note It only goes to the first successful connection, and not best
 *       connection, as in the current build the webserver's size isn't
 *       really known yet. [09 Mar 2026]
 */
void WifiSetup();

/**
 * @brief Sets up the connection to the NTP time pool.
 *
 * @note We need to do this, since there was an initial bug where new
 *       logged items where set to be registered from the unix time epoch
 */
void NTPSetup();

/**
 * @brief Sets up the connection to the ntfy.sh topic.
 */
void NTFYSetup();

/**
 * @brief Ensures that the inventory file, and all related files are available on the SD
card
 *       at the time of boot.
 */
void SDSSetup();

/**
 * @brief Entry notification that is sent on the booting of the system.
 */
void BootNotify();

/**
 * @brief Handles all the communication with the Arduino Uno unit.

```

```

*/
void MessageHandler(SoftwareSerial *serial);

// SetupFunctions.cpp

#include "SetupFunctions.h"

#include "CSV_Parser.h"
#include "SoftwareSerial.h"
#include "Updater.h"
#include "nodemcu/CsvOperations.h"
#include "nodemcu/NtfyNotifier.h"
#include "wl_definitions.h"
#include <Arduino.h>
#include <ESP8266WiFi.h>
#include <IPAddress.h>

#include ".common/comm.h"
#include <NTPClient.h>
#include <WiFiUdp.h>

#include <SD.h>

#include "sensitive_data.cpp"
// Contains defines for:
//      -> NTFY_SERVER
//      -> NTFY_PORT
//      -> NTFY_TOPIC
//      -> pass_list: Array of wifi_pass_t (as defined in .common/comm.h)

#include "SetupFunctions.h"

WiFiUDP ntp_udp;
NTPClient time_client(ntp_udp, "pool.ntp.org", 0, 0);

void WifiSetup()
{
    for (wifi_pass_t wifi_pass : pass_list)
    {
        d_SerialPrint("Connecting to...");
        d_SerialPrintV(wifi_pass.ssid);

        WiFi.begin(wifi_pass.ssid, wifi_pass.pass);

        while ((WiFi.status() != WL_CONNECTED) and (WiFi.status() != WL_WRONG_PASSWORD))
        {
            d_delay(500);
            d_SerialPrint(".");
        }

        switch (WiFi.status()) // Using a switch so that other cases can be handled in the
        future
        {

```

```

    case WL_CONNECTED:
        d_SerialPrintln("\nWiFi connected");
        d_SerialPrint("IP address: ");
        d_SerialPrintlnV(WiFi.localIP());
        return;
        // Kill the function on connecting
    case WL_WRONG_PASSWORD:
        d_SerialPrintf("\nWrong Password for \"%s\"", wifi_pass.ssid);
        continue;
    default:
        continue;
}
}
}

```

```

void NTPSetup()
{
    time_client.begin();

    d_SerialPrintln("Waiting for NTP...");
    while (!time_client.update())
    {
        d_SerialPrint(".");
        delay(500);
    }

    time_t epochTime = time_client.getEpochTime();
    timeval tv = {epochTime, 0};
    settimeofday(&tv, nullptr);
    d_SerialPrintf("Current Unix time: %lld\n", time(nullptr));
}

```

```

void NTFYSetup()
{
    IPAddress ntfy_ip;

    d_SerialPrintf("Resolving host :%s\n", NTFY_SERVER);
    if (WiFi.hostByName(NTFY_SERVER, ntfy_ip) == 0)
    {
        d_SerialPrintln("Fatal: DNS Resolution failed for ntfy.sh!");
        return;
    }
}

```

```

void SDSSetup()
{
    time_client.update();

    if (!SD.begin(D8))
    {
        d_SerialPrintln("SD Init Failed!");
        return;
    }
}

```

```

d_SerialPrintln("SD Card Init'ized.");

if (!SD.exists(INVENTORY_FILE))
{
    d_SerialPrintln("File not found! Creating...");
    if (File f = SD.open(INVENTORY_FILE, FILE_WRITE))
    {
        f.println(""); // Since we're ditching headers now.
        f.close();
    }
    d_SerialPrintln("File Created.");
}
else
    d_SerialPrintln("File Exists. Continuing...");

if (SD.exists(INVENTORY_FILE_TEMP))
    SD.remove(INVENTORY_FILE_TEMP);
}

void MessageHandler(SoftwareSerial *serial)
{
    static char buffer[128];
    static size_t idx = 0;
    static bool waiting_for_csv = false;
    static unsigned long last_byte_time = 0;
    while (serial->available())
    {
        const char c = serial->read();
        last_byte_time = millis();
        if (c == MSG_TERMINATOR)
        {
            if (idx == 0)
                continue;
            buffer[idx] = '\0';
            d_SerialPrint("RX: ");
            d_SerialPrintln(buffer);
            if (waiting_for_csv)
            {
                UpdateCSV(buffer);
                waiting_for_csv = false;
            }
            else if (strcmp(buffer, ReqActBox) == 0)
            {
                d_SerialPrintln("RAB received");
                WritePhysicals(serial);
            }
            else if (strcmp(buffer, UpdBoxes) == 0)
            {
                d_SerialPrintln("UPD received, sending OK");
                serial->print("OK");
                serial->print(MSG_TERMINATOR);
                waiting_for_csv = true;
            }
        }
    }
}

```

```

}
else if (strcmp(buffer, "IVC") == 0)
{
    d_SerialPrintln("IVC received");
    uint16_t count = InventoryCount();
    serial->print("IVC,");
    serial->print(count);
    serial->print(MSG_TERMINATOR);
    d_SerialPrint("IVC Sent back: IVC,");
    d_SerialPrintln(count);
}
else if (strncmp(buffer, "IVW,", 4) == 0)
{
    d_SerialPrintln("IVW received");
    char *save = nullptr;
    char *start_tok = strtok_r(buffer + 4, ",", &save);
    char *rows_tok = strtok_r(nullptr, ",", &save);
    uint16_t start = start_tok ? (uint16_t)atoi(start_tok) : 0;
    uint8_t rows = rows_tok ? (uint8_t)atoi(rows_tok) : 2;
    if (rows == 0)
        rows = 2;
    WriteInventoryWindow(serial, start, rows);
}
else
{
    d_SerialPrintln("Unknown message");
}
idx = 0;
}
else
{
    if (idx < sizeof(buffer) - 1)
        buffer[idx++] = c;
    else
        idx = 0;
}
}
if (idx > 0 && millis() - last_byte_time > 200)
    idx = 0;
}

```